

University Student Management System

Agile Sprint Plan Documentation

PROJECT AGILE TIMELINE ROADMAP

Sprint 0

Week 1

Sprint 1

Week 2–3

Sprint 2

Week 4–5

Sprint 3

Week 6–7

Sprint 4

Week 8–9

Sprint 5

Week 10–13

Sprint 6

Week 14

Sprint 0 — Project Planning & Analysis

Duration: Week 1

Sprint Goal: Establish the project foundation, understand the requirements, define the system structure, and prepare the development environment.

TASKS

- Analyze the project requirements.
- Define the project scope and objectives.
- Identify system actors and roles:
 - Student
 - Instructor
 - College Admin
 - University/System Admin
- Define functional requirements.
- Define non-functional requirements.
- Create Use Case descriptions.
- Design the initial Entity Relationship Diagram (ERD).
- Identify the main database entities and relationships.
- Design the initial UI/UX and wireframes.
- Define the system design system, colors, typography, and reusable components.
- Create the GitHub repository.
- Set up the project structure.
- Set up the development environment.
- Define the team workflow and Git branching strategy.

✓ SPRINT DELIVERABLES

- Project Scope
- Use Case Diagram
- Initial UI/UX Design
- Development Environment
- Requirements
- Initial ERD
- GitHub Repository

Sprint Goal: Implement secure authentication, user roles, authorization, and the basic application layout.

TASKS

- Create the user database structure.
- Create user authentication.
- Implement Login.
- Implement password validation.
- Create the Login Page.
- Create the Welcome Screen.
- Create the main application layout.
- Create the Sidebar.
- Create the Top Navigation Bar.
- Create the User Profile component.
- Implement role-based redirection after login.
- Test authentication and authorization.

✓ SPRINT DELIVERABLES

- | | |
|------------------------|----------------------------------|
| • Working Login System | • Role-Based Access |
| • Welcome Screen | • Main Layout |
| • Sidebar | • Navbar |
| • Basic User Profile | • Authentication & Authorization |

Sprint Goal: Allow students to view their academic information, select courses, complete registration, and generate their registration form.

TASKS

- Create Student Dashboard.
- Create Student Profile.
- Create Academic Registration page.
- Display available courses.
- Allow students to select courses.
- Display selected courses.
- Calculate total registered credit hours.
- Implement course enrollment.
- Implement schedule conflict detection.
- Create the registration confirmation process.
- Create Registration Form.
- Display student information on the registration form.
- Display registered courses and credit hours.
- Add Print / Download functionality.
- Test the complete registration workflow.

✓ SPRINT DELIVERABLES

- | | |
|---------------------|--------------------------------------|
| • Student Dashboard | • Student Profile |
| • Course Catalog | • Academic Registration |
| • Course Enrollment | • Schedule Conflict Validation |
| • Registration Form | • Print / Download Registration Form |

Sprint Goal: Enable instructors to manage their courses and schedules and record student attendance.

TASKS

- Create Instructor Dashboard.
- Create Instructor Profile.
- Create My Courses page.
- Display enrolled students.
- Create Instructor Schedule.
- Create Academic Advising page.
- Create course and lecture structure.
- Create Attendance Management.
- Display lectures for each course.
- Display enrolled students for each lecture.
- Allow instructors to mark students as Present or Absent.
- Allow attendance editing only for available/current lectures.
- Make completed lecture attendance read-only.
- Save attendance records.
- Create Student Attendance page.
- Display attendance percentage.
- Test the complete attendance workflow.

✓ SPRINT DELIVERABLES

- | | |
|------------------------|-------------------------|
| • Instructor Dashboard | • Instructor Profile |
| • My Courses | • Academic Advising |
| • Instructor Schedule | • Attendance Management |
| • Student Attendance | • Attendance Details |

Sprint Goal: Implement grade management, GPA calculation, academic results, and student transcripts.

TASKS

- Create Instructor Grades page.
- Display enrolled students for each course.
- Create grade entry interface.
- Add grade components according to the project rules.
- Validate entered grades.
- Calculate total grades.
- Store student grades.
- Implement semester GPA calculation.
- Implement cumulative GPA calculation.
- Create Student Grades page.
- Organize grades by academic term.
- Display course grades and credit hours.
- Display semester GPA.
- Display cumulative GPA.
- Create Transcript page.
- Display academic history.
- Display courses, GPA, and credit hours.
- Add Print Transcript functionality.
- Add Download Transcript functionality.
- Test grade calculation and GPA results.

✓ SPRINT DELIVERABLES

- | | |
|-------------------------------|-------------------------------|
| • Instructor Grade Management | • Grade Entry |
| • Grade Validation | • GPA Calculation |
| • Student Grades | • Cumulative GPA |
| • Academic Transcript | • Print / Download Transcript |

Sprint Goal: Implement university-level and college-level administration features for managing academic data, exams, results, and statistics.

University / System Admin Tasks

- Create University Admin Dashboard.
- Display university-wide statistics.
- Create Colleges Management (Add, Edit, View, Activate / Deactivate College).
- Create College Admin Management (Add, Edit, View, Activate / Deactivate College Admin).
- Create College Overview (Display college students, departments, courses, and instructors).

College Admin Tasks

- Create College Admin Dashboard & Profile.
- Create Departments Management (Add, Edit, View, Activate / Deactivate Department).
- Create Courses Management (Display courses by department/level, Display credit hours).
- Create Academic Rules / Credit Hours page (Display academic requirements by level).
- Create Exams Management (Add, Edit Exam, Assign Room, Assign Instructor / Committee).
- Create Results Management (View, Edit / Correct, Approve, Publish Results).
- Create Statistics Dashboard (Display department/level stats, passed/failed, success rates).

✓ SPRINT DELIVERABLES

- | | |
|------------------------------|-------------------------|
| • University Admin Dashboard | • College Management |
| • College Admin Management | • College Overview |
| • College Admin Dashboard | • Department Management |
| • Course Management | • Academic Rules |
| • Exam Management | • Results Management |
| • Statistics Dashboard | |

Sprint Goal: Integrate all system modules, perform complete testing, fix issues, finalize documentation, and prepare the final project demonstration.

TASKS

- Integrate all system modules.
- Test Instructor workflows.
- Test University Admin workflows.
- Test authentication and authorization.
- Test schedule conflict detection.
- Test grades.
- Test transcript generation.
- Test results management.
- Perform API testing.
- Perform responsive design testing.
- Improve system performance.
- Review validation and error handling.
- Complete project documentation.
- Prepare final presentation.
- Test Student workflows.
- Test College Admin workflows.
- Test role-based permissions.
- Test course registration.
- Test attendance.
- Test GPA calculations.
- Test exam scheduling.
- Perform database testing.
- Perform UI testing.
- Fix discovered bugs.
- Review UI/UX consistency.
- Prepare test cases and test results.
- Review GitHub repository and commit history.
- Prepare final system demonstration.

✓ SPRINT DELIVERABLES

- Fully Integrated System
- Bug-Free / Stable Release
- Final Documentation
- Installation Guide
- Final Presentation
- Tested System
- Test Cases & Results
- User Manual
- GitHub Repository
- Final Project Demo

SPRINT SUMMARY OVERVIEW

Sprint	Duration	Main Focus
Sprint 0	Week 1	Planning, Requirements, Analysis & Setup
Sprint 1	Week 2–3	Authentication & User Management
Sprint 2	Week 4–5	Student Registration
Sprint 3	Week 6–7	Instructor & Attendance
Sprint 4	Week 8–9	Grades, GPA & Transcript
Sprint 5	Week 10–13	Administration & Academic Management
Sprint 6	Week 14	Integration, Testing & Finalization

DEFINITION OF DONE

A Sprint is considered Done when all of the following conditions are satisfied:

- The implemented feature is completed.
- Frontend is connected to the Backend.
- Backend is connected to the Database.
- Validation is implemented.
- Errors are handled.
- The feature has been tested.
- The feature is integrated with the existing system.
- Code is committed and pushed to GitHub.
- The feature meets the defined requirements.

Trello Board Labels & Automation

Document Type: Technical Workflow Specification

Domain: Software Engineering Project Management

1. LABELS

The project board uses labels to classify tasks by type and priority.

1.1 Task Type Labels

Label	Purpose
Feature	Used for tasks that implement a new system feature or functionality.
Bug Fixing	Used for identifying and fixing errors or issues in the system.
Testing	Used for tasks related to testing and verification.
UI/UX	Used for interface design, user experience, layouts, and visual components.
Documentation	Used for project documentation, user manuals, guides, and presentation-related tasks.
Integration	Used for tasks involving integration between system modules, frontend, backend, database, APIs, or project components.

1.2 Priority Labels

Label	Purpose
High Priority	Used for tasks that require immediate or important attention.
Medium Priority	Used for tasks with normal priority.
Low Priority	Used for tasks that can be handled after higher-priority tasks.

1.3 Additional Workflow Label

Label	Purpose
Sprint Completed	Green label automatically added when a Sprint Card reaches its due date and is moved to the Done list.

1.4 Bug Label

Label	Purpose
Bug Fixing	Red label used when a bug is identified and requires fixing.

Note: Bug Fixing is used as a task-type/workflow label, while Sprint Completed indicates the completion status of a Sprint Card.

2. AUTOMATION RULES

The project uses Trello Automation to reduce repetitive actions, maintain consistent workflow, and notify the team about important events.

Automation 1 —Complete Card Checklists

TRIGGER	When a card is marked as complete by any team member.
ACTION	Automatically check all items in all checklists on the card.
PURPOSE	Ensures that all checklist items are marked as completed when the main card is completed.

Automation 2 —Weekly Team Review

TRIGGER	Every Saturday.
ACTION	Generate a weekly report containing the project cards using the pattern:  Weekly Team Review
PURPOSE	Provides the team with a regular overview of project progress and current tasks.

Automation 3 —Due Date Reminder

TRIGGER	One day before a card is due.
ACTION	Post the following comment:  Reminder: This card is due tomorrow. Please make sure it is completed on time.
PURPOSE	Reminds team members about upcoming deadlines and helps prevent overdue work.

Automation 4 —Overdue Cards

TRIGGER	When a card becomes overdue.
ACTIONS	1. Post the comment:  This card is overdue. Please review and take action. @board 2. Move the card to the top of the <code>Overdue Cards</code> list.
PURPOSE	Automatically highlights delayed work and notifies the team.

Automation 5 —Sprint Completion

CONTEXT Since each Sprint is represented by a Card, the Sprint Card has its own due date.

TRIGGER When a Sprint Card in **To Do** or **In Progress** becomes due.

ACTIONS

1. Move the Sprint Card to the top of the **Done** list.
2. Add the green **Sprint Completed** label.
3. Post the comment:

 Sprint Completed – Please review the sprint results and prepare for the next sprint. @board

PURPOSE Automatically marks the Sprint as completed when its scheduled period ends and notifies the team to review the Sprint and prepare for the next one.

3. CARD BUTTONS

Card Buttons provide quick actions that team members can perform directly from a card.

Report Bug

ACTIONS

1. Add the red **Bug Fixing** label.
2. Post the comment:

 Bug identified. Please review and fix this issue. @board

PURPOSE Provides a quick way to report a problem and move the card into the bug-fixing workflow.